

Trainee C# Technical Test

Part 1 – Planning & Design

Task: Building a task manager console app

Brief Description of How the App Works

This is a simple task manager app which helps people stay organized with their daily tasks. You can add new tasks with a short description and a due date. It tracks which tasks are pending and which ones you've completed. You can check off tasks as you go, and the app will remember your tasks even after you close it, so nothing gets lost.

Class List & Design

List of Classes and Their Responsibilities

1. *Task*

Responsibility: Stores the information for a single task.

Contains:

- Title (string)
- Description (string)
- Due date (DateTime)
- IsCompleted (Boolean)
- TaskPriority (string)

2. *TaskManager*

Responsibility: Manages all task-related methods.

Includes:

- Adding new tasks
- Listing tasks (both completed and pending)
- Marking tasks as complete
- Editing existing tasks
- Saving/loading tasks from a file

3. *Program*

Responsibility: Acts as the main entry point and handles user interaction.

Includes:

- Displaying the main menu
- Reading user input
- Calling the right TaskManager methods

Sketch of the Main Menu and User Interactions

Main Menu – Pseudocode

- 1 - Add a new task
- 2 - List all tasks
- 3 - List completed tasks
- 4 - List pending tasks
- 5 - Mark a task as complete
- 6 - Edit a task
- 7 - Save tasks to file
- 8 - Load tasks from file
- 9 - Exit

Main Loop – User Interactions

Welcome to your Task Manager!

> What would you like to do? [list|add|complete|edit|list-complete|listpending|save|load|quit]

> list

> TODO: show all tasks (with status)

> add

> TODO: ask for title, description, due date

> complete

> TODO: list tasks, ask which to mark as complete

> edit

> TODO: list tasks, ask which to edit, allow changing fields

> list-completed

> TODO: show only completed tasks

> list-pending

> TODO: show only pending tasks

> save

> TODO: save tasks to file

> load

> TODO: load tasks from file

> quit

> Goodbye!

How I Will Save Tasks Between Sessions

- To keep tasks saved even after closing the program, I will store them in a file on the user's computer.
- I will use a JSON file because it's easy to read and write structured data like tasks.
- Each task will have a title, description, due date, and whether it's done or not.
- When the user saves, the program will convert all the tasks into JSON format and save them to the file.
- When the user opens the program again and loads tasks, it will read the JSON file and turn the data back into task objects.
- This way is simple and works well for a console app without needing a database.

Explain Your Reasoning

Why did you choose this structure?

I chose this structure because it separates the data (Task), logic (TaskManager), and user interface (Program), which makes the code easier to manage. I'm using JSON for saving tasks because I learned it at Le Wagon, and it's simple and effective for a console app without needing a database.

What challenges do you anticipate?

Challenges might include handling file errors like missing files, making sure dates are saved and loaded correctly, and validating user input to avoid crashes or invalid data.

How would you test your application?

I would test by adding tasks, marking them complete, listing tasks, and saving/loading to ensure everything works as expected. I'd also test entering invalid inputs like wrong dates or empty titles to make sure the app handles errors properly.